

Questão 3: Testes Práticos e Análise Conceitual

Disciplina: Redes de Computadores

Aluno: João Marcos Sousa Rufino Leal

1. Ordem de Inicialização

O que ocorreu nos testes práticos

Durante os testes, fiz o procedimento de rodar primeiro o script do cliente mantendo o respectivo servidor desligado, tanto no UDP quanto no TCP, e a resposta das duas aplicações foi bem contrastante:

- **Com UDP (`cliente_udp.py`):**
O cliente rodou sem qualquer problema. Ele pediu o texto no terminal, eu digitei, apertei Enter e o programa finalizou indicando envio com sucesso, sem acusar falha alguma na tela — mesmo com a porta do servidor completamente inativa.
 - **Com TCP (`cliente_tcp.py`):**
O comportamento foi o oposto. O cliente nem chegou a pedir mensagem: assim que executou a linha do `connect()`, o programa travou de imediato e estourou um `ConnectionRefusedError`, avisando que a conexão foi rejeitada no destino.
-

Entendendo a diferença técnica (Conexão e Handshake)

A explicação para isso está na filosofia de projeto de cada protocolo da camada de transporte:

1. UDP (Protocolo Não Confiável e Sem Conexão):

O UDP funciona no modelo de datagramas independentes. A chamada `sendto()` não tenta descobrir previamente se tem alguém apto a receber do outro lado. O socket pega os bytes da mensagem, monta o cabeçalho UDP com a porta `6789` e repassa para a camada IP despachar na rede. Para o Python, assim que o pacote sai do buffer local da máquina, a obrigação do cliente está cumprida. Como não há nenhum mecanismo de confirmação ou diálogo prévio, o fato de não existir um processo ouvindo no destino não impede o envio; o pacote simplesmente é descartado na chegada pelo sistema operacional, de forma silenciosa para o cliente.

2. TCP (Protocolo Confiável e Orientado à Conexão):

O TCP trabalha com um canal virtual persistente e não transfere nenhum byte de dados da aplicação antes que a comunicação esteja formalmente autorizada e aberta entre as duas pontas. Para isso acontecer, é obrigatório passar pelo **Three-Way Handshake**:

- O cliente envia um pacote inicial com a flag **SYN** tentando sincronizar com a porta `65432`.
- O sistema operacional da máquina de destino recebe essa solicitação, consulta sua tabela interna e constata que não há nenhum processo registrado em escuta (`listen`) naquele momento.
- Seguindo o padrão do protocolo, a própria pilha de rede do sistema operacional responde rejeitando a tentativa com um pacote **RST** (*Reset*).
- Quando o socket do cliente recebe esse sinal de recusa, o Python imediatamente interrompe o fluxo levantando a exceção `ConnectionRefusedError`.

Dessa forma, enquanto o UDP apenas envia o dado sem se importar com o estado do receptor, o TCP exige que o canal seja estabelecido previamente pelo aperto de mão, acusando erro na hora se o destino estiver inacessível.

2. Concorrência Básica

O segundo cliente tem suas mensagens processadas de imediato?

Não. O segundo cliente não recebe resposta imediata. As mensagens dele só começam a ser processadas pelo servidor quando a sessão do primeiro cliente for encerrada.

Justificativa baseada no funcionamento síncrono do socket

Esse gargalo acontece pela forma como o servidor foi estruturado, utilizando o modelo síncrono padrão (bloqueante) em uma única linha de execução (*single-thread*):

1. Retenção do fluxo de execução:

O código do servidor é linear. Ele chama `server_socket.accept()` para aceitar uma conexão e, logo em seguida, entra em um laço interno `while True` dedicado exclusivamente a esse cliente, ficando bloqueado na chamada `conn.recv(1024)` aguardando mensagens. Enquanto o Cliente 1 mantiver o canal aberto, o ponteiro de execução do Python permanece preso dentro dessa rotina interna.

2. O comportamento do Cliente 2 no sistema operacional:

Quando o Cliente 2 é executado no outro terminal, a chamada `connect()` dele parece funcionar. Isso acontece porque o gerenciamento da conexão em si é feito pelo kernel do sistema operacional: como o socket principal chamou `listen()`, o kernel aceita a *handshake* TCP e coloca o Cliente 2 em uma fila de espera interna (*backlog*).

Entretanto, quando o Cliente 2 envia mensagens, esses dados

ficam apenas parados no buffer de recepção do sistema operacional. O nosso script em Python **ainda não chamou `accept()` para instanciar o socket do Cliente 2**, logo, não há leitura nem devolução de eco. O terminal do Cliente 2 parece congelado esperando resposta.

3. Desbloqueio e atendimento:

Assim que o Cliente 1 digita `"sair"` ou encerra a sua janela, o `recv()` dele recebe zero bytes, quebrando o laço interno. O servidor fecha aquele socket de conexão e finalmente retorna para o laço principal, chamando `server_socket.accept()`. Nesse momento, o servidor "pesca" o Cliente 2 que já estava aguardando na fila, consome as mensagens acumuladas no buffer e dispara as respostas de eco.

Soluções comuns em cenários reais

Para contornar essa restrição em sistemas práticos que precisam atender dezenas ou milhares de usuários ao mesmo tempo, costuma-se usar:

- **Modelo com Threads (`threading`):** onde a thread principal apenas executa o `accept()` e repassa imediatamente o cliente para uma nova thread em segundo plano, voltando a ficar livre para o próximo usuário;
- **Multiplexação de Entrada/Saída / I/O Assíncrono (`selectors` ou `asyncio`):** onde o programa monitora múltiplos descritores de sockets em paralelo, respondendo sob demanda apenas a quem tiver enviado dados naquele instante.